Resumo das atividades por membro

Leitor-exibidor de arquivos .class — Software Básico, UnB · C99 / GCC · formato inspirado no jclasslib

Integrante	Matrícula	Parte
Lucas Mendes	241020750	Coordenação, leitura binária e parser
Antonio Coelho	241038100	Pool de constantes e access flags
Gabriel Pinto	241002331	Bytecode
Leonardo Mileo	241032690	Atributos e dump hexadecimal
Daniel Morais	242042396	Seções da saída e ponto de entrada

Lucas Mendes — coordenação, leitura binária e parser

- **Leitura binária**: leitura do arquivo inteiro para a memória e leitura big-endian de 1, 2 e 4 bytes com checagem de limites — ler além do fim do arquivo marca erro em vez de acessar memória inválida.
- **Parser**: estruturas do .class conforme a JVMS §4 e leitura, na ordem do formato, de magic, versões, pool de constantes (Long/Double ocupando dois slots), flags, this/super, interfaces, fields, métodos e atributos.
- Atributos guardados como bytes brutos após o parse, para serem decodificados só na exibição (um atributo desconhecido nunca quebra a saída).
- Mensagens de erro claras para arquivo inválido: magic number errado (ex.: o cafebabe.class), arquivo truncado, tag desconhecida; liberação de toda a memória alocada.
- **Coordenação**: arquitetura e divisão em módulos, compilação e testes automatizados, analisadores (AddressSanitizer, cppcheck e libFuzzer), integração das partes e conferência da saída com `javap -v -p` em 24 classes.

Antonio Coelho — pool de constantes e access flags

- Exibição de todas as tags do pool no estilo do jclasslib: índice `[n]`, nome da tag e campos de cada entrada.
- Resolução de referências: cada referência mostra `cp #índice` e a constante resolvida entre `< >`; Methodref/Fieldref como `classe.nome : descritor`. Essas funções são reaproveitadas pelas demais partes.
- Constantes numéricas: Integer, Float, Long e Double com os bytes do arquivo e o valor montado a partir dos bits (IEEE 754), incluindo NaN e infinitos; Float com 7 algarismos significativos e Double com 15.
- Strings Utf8 com escape de `\n`, `\t`, `\r` e do caractere nulo.
- **Access flags**: tabelas de flags por contexto (classe, field, método, classe interna) e conversão para texto, ex.: `0x0019 [public static final]`.

Gabriel Pinto — bytecode

- Tabela com os 202 opcodes da JVMS, cada um com mnemônico e formato dos operandos.
- Disassembler do bytecode dos métodos: cada instrução com o pc e o mnemônico.
- Operandos que apontam para o pool (`new`, `ldc`, `invoke*`, `anewarray`…) exibem `#índice` e o valor resolvido; `newarray` mostra o tipo primitivo.
- Desvios com destino absoluto e offset relativo, ex.: `if_icmpge 48 (+25)`, `goto 20 (-25)`.
- `tableswitch` e `lookupswitch`: padding contado a partir do início do código, cada caso com destino e offset, além do default.
- `wide`: índices de 16 bits em loads/stores e `wide iinc` (testado com um método de 300 variáveis locais).

Leonardo Mileo — atributos e dump hexadecimal

- Despacho por nome: o nome do atributo é buscado no pool e associado ao decodificador correspondente; cada atributo mostra nome e tamanho em bytes.
- **Code**: max stack, max locals, tamanho do código, bytecode (via disassembler) e tabela de exceções (início, fim, handler e tipo capturado; `cp #0 = any` para o finally), além dos atributos internos.
- SourceFile, ConstantValue, Signature, Exceptions, LineNumberTable e LocalVariableTable decodificados.
- Atributos não decodificados (StackMapTable, InnerClasses, BootstrapMethods, anotações…) exibidos como dump hexadecimal com offset, 16 bytes por linha.

Daniel Morais — seções da saída e ponto de entrada

- Organização da saída em seções: informações gerais, pool de constantes, interfaces, fields, métodos e atributos da classe.
- Informações gerais: magic number, versões com o nome da versão do Java (ex.: 52 → 1.8), contadores, access flags, this e super class resolvidas.
- Interfaces (índice e nome) e exibição comum de fields e métodos: nome, descritor, flags do contexto e atributos de cada membro.
- Indentação por nível e títulos de seção, para manter a saída alinhada.
- Ponto de entrada do programa: validação dos argumentos, carregamento do arquivo, mensagem de erro em stderr e código de saída 0 (sucesso) ou 1 (erro).